# Arquitectura Operativa Completa

**Intrale Platform** — Issue #1456

**Fecha:** 2026-03-12 | **Documentos:** 4 | **Hooks:** 52+ | **Agentes:** 25+

> **Premisa Principal:** Este reporte documenta cómo interactúan entre sí los agentes IA especializados y cómo se involucran proactivamente en el proceso de desarrollo, desde la planificación inicial hasta el monitoreo post-merge.

## Contenido


- **Documento 1:** Arquitectura Operativa — Visión general, ciclo de vida, interacciones, permisos
- **Documento 2:** Registry de Hooks — Inventario completo de 52+ hooks por evento
- **Documento 3:** Guía de Monitoreo — 3 niveles, umbrales, diagnóstico por subsistema
- **Documento 4:** Troubleshooting — 8 árboles de decisión (A-H) para incidentes

# Documento 1: Arquitectura Operativa

docs/arquitectura-operativa.md — 626 líneas

## 1. Visión General: El Ecosistema de Agentes

El sistema operativo de Intrale Platform está diseñado como un **ecosistema de agentes orquestados** que trabajan juntos de forma **proactiva y autónoma**:

```
┌──────────────────────────────────────────────────────────────┐
│                    ECOSISTEMA DE AGENTES                     │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Usuario solicita feature/bug fix (CLI o Telegram)           │
│         ↓                                                    │
│  Planner: analiza, prioriza, crea sprint-plan.json           │
│         ↓                                                    │
│  Start-Agente.ps1 lanza 3 agentes máximo en paralelo         │
│         ↓                                                    │
│  Agentes especializados trabajan en worktrees aislados:      │
│    • Agent 1 (issue A) implementa feature                    │
│    • Agent 2 (issue B) implementa bug fix                    │
│    • Agent 3 (issue C) documenta cambios                     │
│         ↓                                                    │
│  Al terminar cada agente:                                    │
│    1. Hook Stop dispara agent-concurrency-check.js           │
│    2. Si hay capacidad, promociona siguiente de _queue[]     │
│    3. Start-Agente.ps1 lanza el nuevo agente en paralelo     │
│         ↓                                                    │
│  Mientras trabajan, sistemas monitorean en background:       │
│    • telegram-commander: espera comandos remotos             │
│    • scrum-monitor-bg: verifica salud del sprint c/30min     │
│    • agent-watcher: detecta agentes terminados               │
│    • ci-monitor-bg: pollea GitHub Actions tras cada push     │
│         ↓                                                    │
│  Al finalizar todas las historias del sprint:                │
│    Scrum cierra sprint, genera métricas, notifica velocity   │
└──────────────────────────────────────────────────────────────┘
```

## 2. Ciclo de Vida Completo de un Agente

### Fase 1: PLANIFICACIÓN (Planner)

**Agente involucrado:** `/planner`

1. Planner analiza issues abiertos (prioridad, dependencias, tamaño)
2. Selecciona top N historias accionables (default 7-10)
3. Valida dependencias con `/po` (Product Owner)
4. Crea `scripts/sprint-plan.json` con estructura:
   - `agentes[]` — Primeros 3 (lanzados por Start-Agente.ps1)
   - `_queue[]` — Restantes (4 a N, promovidos automáticamente)
   - `_completed[]` — Historial de agentes terminados
   - `concurrency_limit: 3` — máximo de agentes simultáneos

### Fase 2: EJECUCIÓN PARALELA (Agentes especializados)

**Agentes involucrados:** `/backend-dev`, `/android-dev`, `/ios-dev`, `/web-dev`, `/desktop-dev`, `/ux`, `/qa`, `/security`

Cada agente:

1. Lee su issue con `gh issue view <N>`
2. Invoca pipeline de especialistas ( `/guru` → `/review` → `/builder` → `/tester` )
3. Trabaja en worktree aislado ( `platform.agent-<issue>-<slug>` )
4. Hace cambios, corre tests, valida build
5. Invoca `/delivery` para commit + PR + push

> **Interacción clave:** Los agentes pueden invocar otros agentes en el pipeline:
> - Agent implementa con `/backend-dev` → invoca `/builder` para validar → `/tester` para tests
> - Si falla, agente pide ayuda a `/guru` para investigar el error
> - Al terminar, `/review` valida el código antes de `/delivery`

### Fase 3: PROMOCIÓN AUTOMÁTICA

**Disparadores:**

- **Hook Stop** — `agent-concurrency-check.js` (ejecutado cuando agente termina)
- **Watcher externo** — `agent-watcher.js` (cada 60s, detecta agentes muertos)

```
Agent 1 termina (git push finalizado)
    ↓
Hook Stop → agent-concurrency-check.js
    ↓
Busca agentes en lista:
  • Cuenta activos (status != "waiting"): 2
  • Límite: 3 → Capacidad disponible: 1 slot
    ↓
¿Hay items en _queue[]?
  ✓ SÍ → Promocionar siguiente de _queue[] a agentes[]
    ↓
Actualizar sprint-plan.json → Lanzar Start-Agente.ps1
```

### Fase 4: MONITOREO PASIVO (4 subsistemas en background)

| Subsistema | Ciclo | Propósito | Proactividad |
|---|---|---|---|
| **telegram-commander** | 10-300s (adaptativo) | Recibir comandos remotos via Telegram | Envía heartbeat con screenshots del Monitor |
| **scrum-monitor-bg** | Cada 30min | Verificar salud del sprint, auto-reparar | Genera métricas automáticas sin que se lo pida |
| **agent-watcher** | Cada 60s | Detectar agentes muertos, promover cola | Corre continuamente sin trigger externo |
| **ci-monitor-bg** | Cada 30s (por push) | Pollear GitHub Actions, notificar resultado | Lanzado automáticamente por post-git-push.js |

## 3. Matriz de Interacciones Proactivas

| Agente A | Invoca | Agente B | Propósito | Momento |
|---|---|---|---|---|
| **Planner** | → | `/po` (dependencias) | Validar orden de ejecución | Pre-sprint |

| | | | | |
|---|---|---|---|---|
| **Planner** | → | `/historia` | Crear nuevas historias si hay gaps | Pre-sprint |
| **Any Agent** | → | `/branch` | Crear rama antes de trabajar | Inicio |
| **Any Agent** | → | `/po` (acceptance) | Entender criterios del issue | Inicio |
| **Any Agent** | → | `/guru` | Investigar patrones/libs | Cuando no sabe |
| **BackendDev** | → | `/builder` | Validar build backend | Pre-delivery |
| **BackendDev** | → | `/tester` | Ejecutar tests unitarios | Pre-delivery |
| **Any Dev Agent** | → | `/security` | Auditoría de seguridad | Pre-delivery |
| **Any Dev Agent** | → | `/review` | Code review automatizado | Pre-delivery (gate) |
| **Any Dev Agent** | → | `/qa` | Tests E2E en entorno real | Pre-delivery (si UI) |
| **Any Dev Agent** | → | `/delivery` | Commit + PR + push | Final |
| **post-git-push** | → | ci-monitor-bg | Pollear CI tras push | Después de push |
| **agent-concurrency-check** | → | Start-Agente.ps1 | Lanzar siguiente agente | Tras Stop |
| **agent-watcher** | → | Start-Agente.ps1 | Promocionar cola (fallback) | Cada 60s |
| **scrum-monitor-bg** | → | `/scrum` (auto-repair) | Reparar inconsistencias | Cada 30min |
| **telegram-commander** | → | `/monitor` / `/ops` | Responder comandos | On-demand via Telegram |

## 4. Sistema de Permisos

| Severidad | Herramientas | Comportamiento |
|---|---|---|
| AUTO_ALLOW | TaskCreate, TaskUpdate, Skill, EnterPlanMode | Aprobadas automáticamente |
| LOW | WebFetch, WebSearch, Bash genérico | Auto-aprobadas (sin Telegram) |
| MEDIUM | git push, curl POST, Bash con pipes | Auto-aprobadas desde #1302 |
| HIGH | rm -rf, git reset --hard, destructivos | Requiere aprobación manual via Telegram |

## 5. Archivos de Estado Críticos

| Archivo | Propósito | Quién escribe | Quién lee |
|---|---|---|---|
| `scripts/sprint-plan.json` | Plan actual + agentes + cola + métricas | Planner, concurrency-check, watcher | Start-Agente, agentes, Monitor |
| `.claude/activity-log.jsonl` | Log de actividad en tiempo real | activity-logger.js (hook) | Monitor, tendencias |
| `.claude/hooks/health-check-*.json` | Estado de salud por componente | ops-check, health-check | Monitor |
| `.claude/hooks/scrum-health-history.jsonl` | Historial de inconsistencias | scrum-monitor-bg | Análisis tendencias |
| `.claude/sessions/*.json` | Metadatos de sesión Claude | Claude Code | Monitor, session-gc |

## 6. Diagrama de Flujo: 5 Agentes en Paralelo

```
HORA 10:00 — Planner crea sprint-plan.json
  agentes: [#1456, #1457, #1458]  |  _queue: [#1459, #1460, #1461]
     ↓
Start-Agente.ps1 all → lanza 3 agentes + agent-watcher en background

HORA 10:15 — Agent 1 hace git push
  → post-git-push.js marca "waiting", promociona Agent 4 de _queue
  → ci-monitor-bg inicia polling GitHub Actions
  → AHORA: 4 agentes (1 waiting + 3 activos)

HORA 10:45 — Agent 2 termina
  → Hook Stop: agent-concurrency-check.js
  → Cola vacía → sin promoción

HORA 11:00 — scrum-monitor-bg ejecuta (cada 30min)
  → Verifica salud: Agent 2 terminado, PR sin mergear → Auto-mergea

HORA 11:30 — CI de Agent 1 pasa
  → ci-monitor-bg notifica "CI PASS ✓" → Auto-mergea PR

HORA 12:30 — Último agente termina
  → scrum-monitor-bg cierra sprint → velocity = 100%
  → SPRINT CLOSED → Ready para /planner sprint (SPR-026)
```

## 7. Tabla de Eventos y Hooks

| Evento | Cuándo | Hooks | Acción |
|---|---|---|---|
| **Notification** | Siempre | notify-telegram.js | Informar usuario via Telegram |
| **Stop** | Sesión termina | stop-notify, agent-concurrency-check, scrum-monitor-bg | Notificar, promocionar cola, auditar sprint |
| **PreToolUse[Bash]** | Antes de bash | branch-guard, worktree-guard, delivery-gate | Bloquear push a main, alertas |
| **PreToolUse[*]** | Antes de tool | permission-gate | Control de acceso por severidad |
| **PostToolUse[Bash]** | Después de bash | post-git-push | Orquestar CI, promocionar cola |
| **PostToolUse[*]** | Después de tool | permission-tracker, activity-logger | Auditoría y trazabilidad |
| **PermissionRequest** | Solicitud acceso | permission-approver | Auto-aprobar o enviar a Telegram |

# Documento 2: Registry de Hooks — Inventario Completo

docs/operativo/hooks-registry.md — 353 líneas — 52+ hooks registrados

## Event: Notification

| Hook | Propósito | Estado que toca | Timeout | Crítico |
|---|---|---|---|---|
| `notify-telegram.js` | Enviar notificación a Telegram | telegram-outbox.json, telegram-message-registry.json | 15s | NO |

## Event: Stop

| Hook | Propósito | Estado que toca | Timeout | Crítico |
|---|---|---|---|---|
| `stop-notify.js` | Notificar fin de agente | .claude/sessions/*.json | 15s | NO |
| `post-console-response.js` | Registrar output final | activity-log.jsonl, hook-debug.log | 10s | NO |
| `agent-concurrency-check.js` ★★★ | Promocionar siguiente de _queue[] | sprint-plan.json, hook-debug.log | 20s | **SÍ** |
| `scrum-monitor-bg.js` ★ | Auditar salud del sprint | sprint-plan.json, scrum-health-history.jsonl | 60s | NO |
| `pr-cleanup.js` | Limpiar PRs viejas | Git branches remotas | 120s | NO |

## Event: PreToolUse

### Matcher: "Bash"

| Hook | Propósito | Lógica | Crítico |
|---|---|---|---|
| `branch-guard.js` ★★ | Bloquear git push en main | Detecta `git push origin main` → Error fatal | **SÍ** |
| `worktree-guard.js` ★ | Alerta en rama protegida | Si main/develop → Alerta Telegram | NO |
| `delivery-gate.js` ★★ | Bloquear delivery incompleta | Cambios sin stagear, tests rotos, build roto | **SÍ** |

### Matcher: "" (todas las herramientas)

| Hook | Propósito | Severidades | Timeout |
|---|---|---|---|
| `permission-gate.js` ★★★ | Control de acceso por severidad | AUTO_ALLOW → LOW → MEDIUM → HIGH | 1830s |

## Event: PostToolUse

### Matcher: "Bash"

| Hook | Propósito | Lógica | Crítico |
|---|---|---|---|
| `post-git-push.js` ★★★ | Orquestar CI tras push | Detecta git push → marca waiting → promociona cola → lanza ci-monitor-bg | NO |

### Matcher: "" (todas)

| Hook | Propósito | Crítico |
|---|---|---|
| `ensure-permissions.js` | Registrar permiso otorgado | NO |
| `permission-tracker.js` | Registrar cambios de permisos | NO |
| `activity-logger.js` ★ | Registrar TODA actividad en activity-log.jsonl | NO |

## Event: PermissionRequest

| Hook | Propósito | Timeout |
|---|---|---|
| `permission-approver.js` | Auto-aprobar o enviar a Telegram | Hasta respuesta usuario |

## Subsistemas Background

| Script | Lanzado por | Ciclo | PID file | Propósito |
|---|---|---|---|---|
| `agent-watcher.js` ★★★ | Start-Agente.ps1 all | 60s | agent-watcher.pid | Detectar agentes muertos, promover |
| `ci-monitor-bg.js` ★★ | post-git-push.js | 30s | ci-monitor-bg.pid | Pollear CI, notificar, auto-merge |
| `scrum-monitor-bg.js` ★ | Hook Stop | 30min | scrum-monitor-bg.pid | Auditar sprint, auto-reparar |
| `telegram-commander.js` ★★ | commander-launcher.js | 10-300s | telegram-commander.lock | Comandos remotos, heartbeat |

## Dependencias Entre Scripts

```
agent-concurrency-check.js
  ├─→ sprint-sync.js (sincronizar roadmap)
  ├─→ Start-Agente.ps1 (lanzar siguiente)
  └─→ notify-telegram.js (notificar)

post-git-push.js
  ├─→ ci-monitor-bg.js (lanzar en background)
  ├─→ Start-Agente.ps1 (promover cola)
  └─→ notify-telegram.js (notificar)

scrum-monitor-bg.js
  ├─→ health-check-sprint.js (auditoría)
  ├─→ scrum-auto-corrections.js (auto-reparar)
  └─→ notify-telegram.js (alerta crítica)

permission-gate.js
  ├─→ permission-approver.js (decisión)
  └─→ telegram-commander.js (aprobación remota)
```

# Documento 3: Guía de Monitoreo

docs/operativo/monitoring-guide.md — 457 líneas

## Tres Niveles de Monitoreo

| Nivel | Método | Frecuencia | Acción del usuario |
|---|---|---|---|
| PASIVO | Telegram Heartbeat | Cada 10-30 min (automático) | Solo leer notificaciones |
| ACTIVO | Comandos Remotos | On-demand | Enviar `/monitor`, `/ops`, `/cleanup` |
| INTERNO | Logs y State Files | Manual (debugging) | Leer activity-log.jsonl, hook-debug.log |

## Monitoreo por Subsistema

### Agentes (concurrency, promotion, lifecycle)

**Qué monitorear:** ¿Hay siempre 3 agentes activos? ¿Se promueven de _queue[]? ¿Hay agentes en "waiting"?
**Cómo:** `/monitor` en Telegram o `cat scripts/sprint-plan.json | jq '.agentes[]'`
**Auto-reparación:** agent-watcher detecta zombies (30+ min) → marca dead → promueve siguiente

### CI Monitor (GitHub Actions)

**Qué monitorear:** ¿Builds se lanzan tras push? ¿Notifica status? ¿Auto-merge funciona?
**Cómo:** Observar mensajes Telegram: "CI: Testando..." / "CI: PASS ✓" / "CI: FAIL ✗"
**Auto-reparación:** ci-monitor-bg reintenta polling cada 30s; si falla >5 veces → alerta Telegram

### Sprint Health (Project V2)

**Qué monitorear:** ¿Issues en estado correcto? ¿PRs mergeados = issues Done?
**Cómo:** `/scrum audit` en Telegram
**Auto-reparación:** scrum-monitor-bg detecta inconsistencias cada 30min → auto-repara las menores

### Permisos y Seguridad

**Qué monitorear:** ¿HIGH severity requiere aprobación? ¿Hay intentos bloqueados?
**Cómo:** `tail -50 .claude/hooks/permission-tracker.log`
**Umbrales:** >5 bloqueos en 10min → posible error recurrente

## Tabla de Umbrales

| Métrica | Amarillo (≤) | Rojo (≤) | Acción |
|---|---|---|---|
| Agentes activos | <2 cuando esperado ≥3 | 0 durante sprint | `/ops` + reinicio |

| | | | |
|---|---|---|---|
| Agente en "waiting" | >15 min | >30 min | Revisar CI, cancelar workflow |
| Cola pendiente | >5 items | >10 items | Sprint demasiado grande |
| CI en "running" | >30 min | >60 min | Cancelar workflow en GitHub |
| Scrum inconsistencies | >1 | >3 | `/scrum audit` + auto-repair |
| hook-debug.log size | >1 MB | >2 MB | `/cleanup --logs --run` |
| Telegram no responde | >5 seg | >30 seg | Reiniciar commander |

# Documento 4: Troubleshooting — Árboles de Decisión

docs/operativo/troubleshooting.md — 582 líneas — 8 flujos (A-H)

## Flujo Principal: Identificar el Síntoma

| Síntoma | Flujo | Descripción |
|---|---|---|
| Agente no se lanza / no continúa | **A** | Rama, concurrencia, promoción, Start-Agente |
| Build/Tests fallan | **B** | Compilación Kotlin, tests, dependencias Gradle |
| CI/GitHub stuck | **C** | PR sin crear, workflow stuck, merge fallido |
| Permisos bloqueados | **D** | branch-guard, HIGH severity, aprobación Telegram |
| Telegram no responde | **E** | Commander muerto, token, red, chat ID |
| Sprint desincronizado | **F** | PR merged/issue open, plan ≠ Project V2 |
| Error desconocido | **G** | Diagnóstico general, /ops, /guru, logs |
| Workspace lento / disco lleno | **H** | Disco, worktrees, procesos zombie, logs grandes |

## FLUJO A: Agente no se lanza

**Pregunta 1:** ¿La rama `agent/*` existe? → NO: `/branch <issue> <slug>`
**Pregunta 2:** ¿Hay 3+ agentes activos? → SÍ: Esperar (límite concurrencia)
**Pregunta 3:** ¿Hay items en _queue[]? → SÍ: Verificar hook Stop → Verificar Start-Agente.ps1 → Lanzar manualmente
**Último recurso:** `/ops --fix` + reintentar lanzamiento manual

## FLUJO B: Build/Tests fallan

**Compilación Kotlin:** Leer error → buscar en logs → `/guru` para investigar → arreglar → `/builder`
**Test assertion:** Leer esperado vs actual → `/guru` → arreglar test O código → `/tester`
**Dependencia Gradle:** `./gradlew clean build` → verificar red → `/guru`

## FLUJO C: CI/GitHub stuck

**Sin PR:** Ejecutar `/delivery`
**Workflow no trigger:** Verificar .github/workflows/ → re-trigger en web
**Workflow stuck >30min:** `gh run cancel <run-id>` + re-run
**CI pass pero no merge:** `gh pr merge <PR> --auto`

## FLUJO D: Permisos bloqueados

**git push main:** branch-guard (correcto) → crear rama con `/branch`
**rm -rf:** HIGH severity → aprobar en Telegram o usar `/cleanup --run`
**Agente pide comando riesgoso:** Denegar → investigar → parar agente

## FLUJO E: Telegram no responde

**Commander muerto:** Matar PID → borrar lock → relanzar `commander-launcher.js`
**Token inválido:** Obtener nuevo en @BotFather → actualizar telegram-config.json
**Red bloqueada:** `curl -I https://api.telegram.org/` → verificar VPN/firewall
**Chat ID incorrecto:** Enviar /start al bot → obtener chat ID correcto

## FLUJO F: Sprint desincronizado

**PR merged + issue open:** `node health-check-sprint.js --auto-repair`
**Issue Done sin PR:** Buscar rama → crear PR o revisar historial
**Agent done + In Progress:** `scrum-auto-corrections.js --repair-status`
**Plan ≠ Project V2:** Decidir fuente de verdad → `sprint-sync.js --force-sync`

## FLUJO G-H: Error desconocido / Performance

**Error desconocido:** `/ops` → `/guru "<error>"` → buscar en activity-log → crear issue si persiste
**Disco lleno:** `/cleanup --run --deep`
**Worktrees antiguos:** `/cleanup --worktrees --run`
**Procesos zombie:** Identificar con `ps aux` → matar PIDs viejos
**Logs gigantes:** `/cleanup --logs --run`

## Referencia Rápida

| Problema | Comando Diagnóstico | Acción Rápida |
|---|---|---|
| Agente no se promueve | `grep ConcurrencyCheck hook-debug.log` | `Start-Agente.ps1 <num>` |
| CI stuck | `gh run view <run-id>` | Cancel + re-run en GitHub |
| Telegram no responde | `ps aux \| grep commander` | Relanzar commander |
| Tests fallan | `./gradlew test --info` | Arreglar test O código |
| Build falla | `./gradlew clean build` | Investigar con `/guru` |
| Permisos bloqueados | Ver Telegram | Aprobar / Denegar |
| Disco lleno | `df -h /c/Workspaces/` | `/cleanup --run --deep` |
| Sprint desincronizado | `/scrum audit` | `/scrum repair --auto` |

**Arquitectura Operativa Completa** — Intrale Platform — Issue #1456

Generado: 2026-03-12 | 4 documentos consolidados | 52+ hooks | 25+ agentes

Documento vivo — Actualizar cuando cambie la arquitectura de agentes o flujo de hooks